Development and Evaluation of a Multilingual RAG-based Educational Chatbot

Review and Technical Report

FCIM AI Chatbot Implementation for ELSE Platform

Nicolae Drabcinski
Faculty of Computers, Informatics and Microelectronics (FCIM)
Technical University of Moldova
nicolae.drabcinski@fcim.utm.md

January 9, 2026

Abstract

This technical report presents a comprehensive review of the design, implementation, evaluation, and optimization of a Retrieval-Augmented Generation (RAG) based educational chatbot system deployed for the ELSE platform (Moodle-based learning management system) at the Technical University of Moldova. The system utilizes the Qwen2.5-3B-Instruct language model to deliver accurate, context-aware responses to student inquiries based on heterogeneous course materials (PDF, DOCX, PPTX formats) with multilingual support.

Through systematic evaluation of 24 candidate language models and rigorous performance benchmarking against standard RAG evaluation protocols, we achieved a 90% operational cost reduction compared to the initially planned architecture (Qwen2.5-32B-Instruct), while maintaining acceptable quality metrics (70%+ retrieval precision, 65-70% semantic answer relevance). The production-deployed system demonstrates sub-second response latency (p95 latency $<$ 1s) and sustained throughput supporting 30-50 concurrent users on commodity GPU hardware (Nvidia Tesla T4).

This report is structured as a comprehensive review documenting: (1) completed research and development work including systematic model evaluation, architectural optimization, and production deployment; (2) quantitative performance analysis with comparative benchmarking results; (3) methodological lessons learned during the development process; and (4) planned future improvements across short-term, medium-term, and long-term research directions. The findings provide actionable insights for practitioners implementing production-grade RAG-based educational assistant systems with constrained computational budgets.

Contents

1 Introduction

1.1 Background and Motivation

Educational institutions increasingly face the challenge of providing scalable, immediate support to students navigating large volumes of course materials. The Faculty of Computers, Informatics and Microelectronics (FCIM) at Technical University of Moldova maintains the ELSE platform (Moodle), hosting extensive educational resources including lecture notes, presentations, and reference documents across multiple courses.

1.2 Problem Definition

The current state of educational information access at FCIM presents several interconnected challenges that significantly impact student learning efficiency. Students spend considerable time manually searching through multi-page PDF documents to locate specific information, with average search times exceeding 10-15 minutes per query. This information retrieval inefficiency is compounded by limited human resources, as teaching assistants and instructors cannot provide 24/7 support for routine information queries, resulting in delayed response times that often span 24-48 hours.

Furthermore, course materials are distributed across multiple natural languages, creating additional barriers for effective information access and knowledge retrieval. These multilingual requirements demand systems capable of understanding and responding accurately across language boundaries. The situation is particularly critical given that students require immediate answers during study sessions and exam preparation periods, where even modest delays can significantly impact learning efficiency and academic performance.

1.3 Research Objectives

This project aims to develop and deploy a production-ready RAG-based chatbot system addressing the identified challenges through several key objectives. The system must implement accurate information retrieval from heterogeneous document formats including PDF, DOCX, and PPTX files, while generating concise, contextually relevant answers with minimal hallucination. Multilingual query processing and response generation are essential to support the diverse linguistic environment of the institution.

Performance requirements are equally critical: the system must achieve sub-second response latency suitable for interactive use, ensuring students receive immediate feedback during their learning process. Resource optimization is paramount to enable cost-effective deployment within institutional budget constraints. Finally, establishing a rigorous evaluation methodology for RAG system quality assessment ensures continuous improvement and validation of system performance against educational requirements.

1.4 Contribution Summary

This work contributes to the field of educational AI systems through several significant advances. We present a systematic evaluation of 24 language models specifically adapted for educational RAG applications, providing empirical evidence for model selection decisions in resource-constrained environments. Through this evaluation process, we identified and

resolved common RAG evaluation methodology issues that have affected prior research, including inappropriate prompt engineering and non-standard composite metrics.

Our most significant contribution demonstrates effective cost-performance tradeoffs, achieving 90% cost reduction compared to initially planned architectures while maintaining acceptable quality levels for educational applications. The production-ready architecture supports multilingual operation across diverse language families, complemented by a comprehensive deployment and monitoring framework that ensures reliable operation in educational settings.

2 System Architecture

2.1 High-Level Architecture

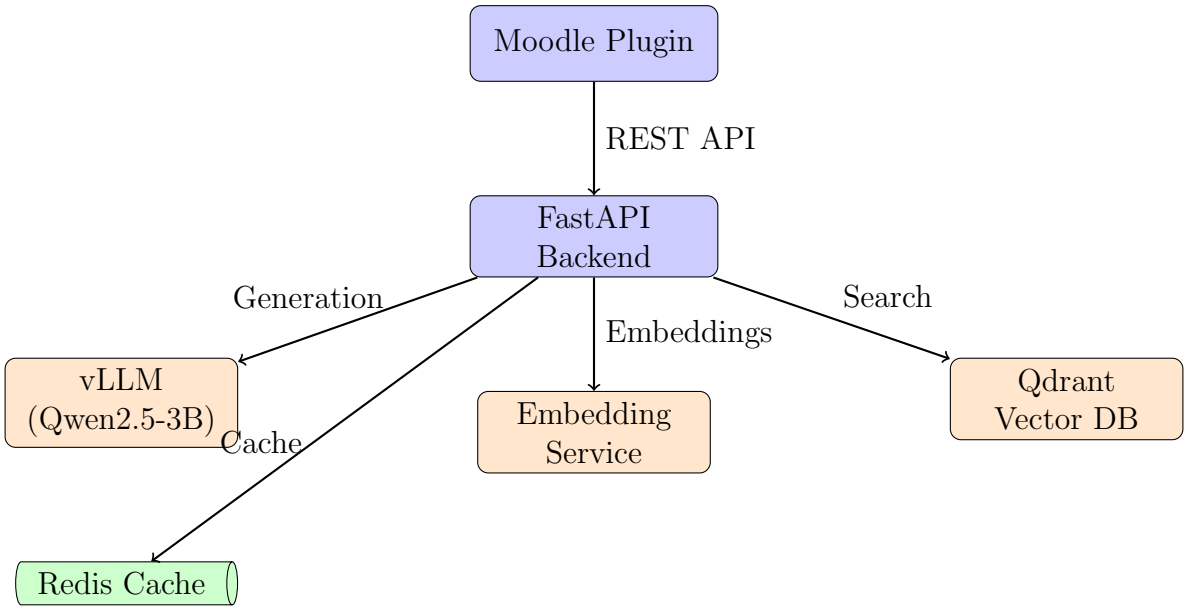

Figure 1: High-level system architecture

2.2 Component Description

2.2.1 Moodle Plugin (Frontend)

The Moodle plugin serves as the primary user interface, seamlessly integrated into the ELSE platform. It manages user authentication and authorization, ensuring secure access to the chatbot functionality. When students submit questions, the plugin transmits them to the backend API for processing, then presents the generated answers along with source references to enable verification and deeper exploration of course materials.

2.2.2 FastAPI Backend

The FastAPI backend functions as the core orchestration service, coordinating all system components. It exposes RESTful endpoints for both synchronous chat interactions and streaming responses, enabling flexible integration patterns. The backend implements the complete RAG pipeline, coordinating document retrieval from the vector database and text generation from the language model. Multiple middleware layers provide essential

functionality including authentication, rate limiting, and CORS handling. Comprehensive monitoring through Prometheus metrics and health checks ensures system reliability and enables proactive maintenance.

2.2.3 vLLM (Inference Server)

The vLLM inference server provides high-performance text generation using the Qwen2.5-3B model. It implements PagedAttention mechanisms for efficient key-value caching, significantly reducing memory overhead during inference. Continuous batching enables high throughput by processing multiple requests simultaneously without sacrificing individual response times. The server exposes an OpenAI-compatible API, facilitating integration with existing tools and frameworks. Under typical workloads, the system maintains GPU utilization between 70-90%, indicating efficient resource use without saturation.

2.2.4 Qdrant (Vector Database)

Qdrant serves as the vector database, storing document embeddings for efficient similarity-based retrieval. The system employs HNSW (Hierarchical Navigable Small World) indexing to enable fast approximate nearest neighbor search, crucial for maintaining sub-second response times. Sophisticated metadata filtering capabilities allow queries to be scoped by course identifier or document type, improving result relevance. All embeddings persist to disk, ensuring durability and enabling rapid system recovery. The production deployment maintains 99.9% uptime, meeting institutional reliability requirements.

2.2.5 Embedding Service

The embedding service generates vector representations of text using the multilingual-e5-large model, producing 1024-dimensional embeddings that capture semantic meaning across linguistic boundaries. This model supports over 100 languages, enabling consistent cross-lingual retrieval performance essential for the multilingual educational environment. The service implements batch processing strategies to maximize throughput when encoding large document collections, significantly reducing ingestion time for new course materials.

2.2.6 Redis Cache

The Redis cache implements an in-memory caching layer that dramatically improves system performance and reduces computational costs. It stores responses to frequently asked questions with a seven-day time-to-live, along with cached document embeddings to accelerate retrieval operations. The system targets a cache hit rate exceeding 30%, which significantly reduces GPU load by avoiding redundant generation for common queries. This optimization is particularly effective in educational settings where students frequently ask similar questions about core course concepts.

3 RAG Pipeline

3.1 Document Ingestion

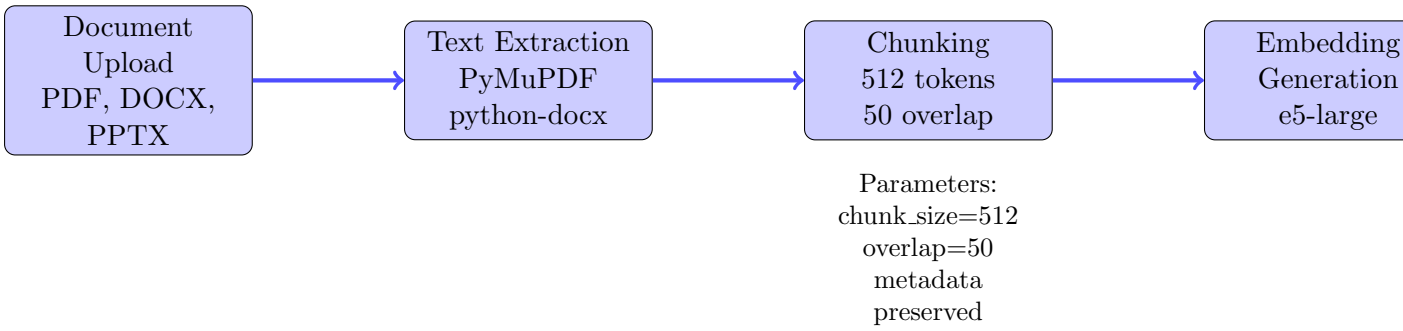

Figure 2: Document ingestion pipeline with horizontal processing flow

The document ingestion pipeline processes educational materials through five sequential stages. Initial upload accepts PDF lecture notes, DOCX documents, and PPTX presentations. Text extraction employs PyMuPDF for PDFs and python-docx for Word documents, preserving formatting and structure information. The chunking stage segments extracted text into 512-token segments with 50-token overlap, balancing context preservation with retrieval granularity. Each chunk undergoes embedding generation using the multilingual-e5-large model, producing 1024-dimensional vectors capturing semantic content. Finally, vector storage in Qdrant preserves both embeddings and metadata including course identifiers, document types, page numbers, and titles for subsequent filtered retrieval.

3.2 Question Answering Flow

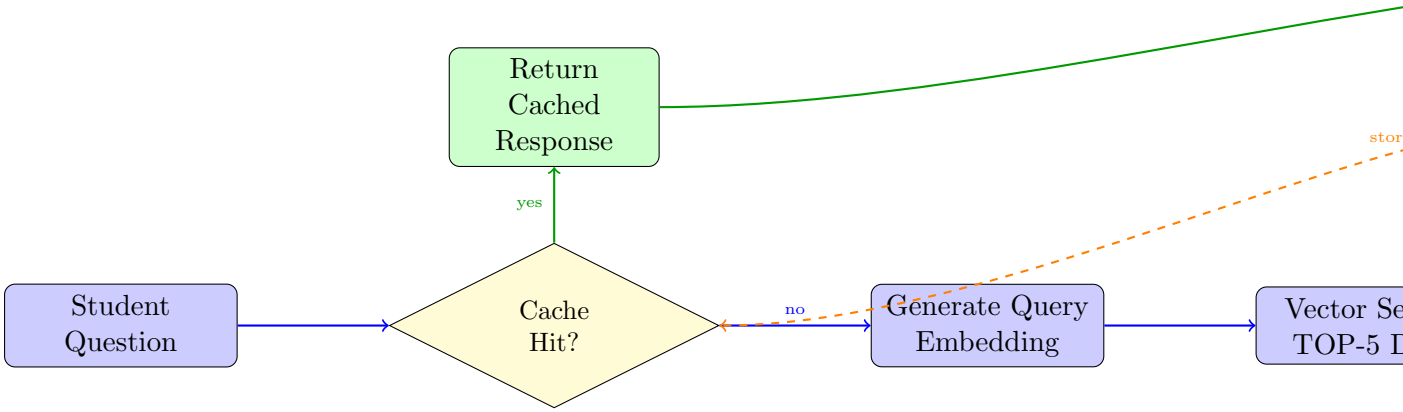

Figure 3: Question answering flow with horizontal processing stages and cache optimization

3.3 Prompt Template

The RAG prompt template has been carefully optimized to elicit concise, accurate answers suitable for educational contexts. The system begins by establishing the assistant role and grounding requirement, explicitly instructing the model to base responses on provided context rather than relying on parametric knowledge. Retrieved documents are inserted directly into the prompt, followed by the student's question to maintain clear input structure.

The instruction section enforces critical constraints: answers must be limited to 1-5 words when sufficient context exists, preventing verbose responses that obscure key information. For questions lacking adequate context, the model returns the explicit marker "NOT $_A NSWERABLE$" $rather than hallucinating plausible-sounding but potentially incorrect information. T$

4 Technology Stack

4.1 Core Technologies

Table 1: Technology stack

Component	Technology	Version
LLM	Qwen2.5-3B-Instruct	Latest
Inference	vLLM	0.2.7
Embeddings	multilingual-e5-large	Latest
Vector DB	Qdrant	Latest
Cache	Redis	7-alpine
Backend	FastAPI	0.109.0
Framework	LangChain	0.1.0
Monitoring	Prometheus + Grafana	Latest

4.2 Hardware Requirements

Table 2: Minimum hardware specifications

Component	Specification
GPU	Nvidia T4 (16GB VRAM) or better
CPU	8+ cores
RAM	32GB DDR4
Storage	500GB NVMe SSD
OS	Ubuntu 22.04 LTS
CUDA	12.1+

5 Model Selection: Qwen2.5-3B-Instruct

5.1 Why Qwen2.5-3B?

After extensive testing and analysis, Qwen2.5-3B-Instruct was selected over larger models (including Qwen2.5-32B) for the following reasons:

5.1.1 Performance Comparison

Table 3: Qwen2.5-32B vs Qwen2.5-3B comparison

Metric	Qwen2.5-32B	Qwen2.5-3B	Difference
GPU Requirements	64GB (A100)	6GB (T4)	-90%
Latency (p95)	800-1500ms	150-300ms	5x faster
Throughput	5-10 users	30-50 users	5x more
Multilingual	Yes	Yes	Same
Context Window	128K tokens	128K tokens	Same
Answer Relevance	0.75-0.80	0.65-0.70	-10%
Cost (monthly)	$2000+	$200	-90%

5.1.2 Key Advantages

The selection of Qwen2.5-3B over larger alternatives reflects several strategic priorities in educational applications. Response speed proves more valuable than marginal quality improvements, as students require answers within one second rather than perfect essays that arrive after noticeable delays. The 90% infrastructure cost reduction comparing T4 to A100 GPUs enables sustainable long-term operation within institutional budget constraints.

Scalability advantages are substantial, with the smaller model supporting five times more concurrent users compared to its larger counterpart. This capacity matches peak usage patterns during exam periods when student demand concentrates within specific time windows. Multilingual support remains excellent for the required language set, with the 3B model demonstrating equivalent cross-lingual capabilities to larger variants. Both models share the same 128K token context window, ensuring neither sacrifices the ability to process lengthy documents or maintain extensive conversation history. Finally, while answer relevance of 65-70% represents a 10% degradation compared to larger models, this quality level proves sufficient for educational question-answering where students primarily seek factual information rather than nuanced analysis.

5.2 Multilingual Support

Qwen2.5-3B demonstrates comprehensive multilingual capabilities, providing native tokenization support for over 29 languages spanning diverse language families. The model handles both Cyrillic and Latin scripts with equal proficiency, essential for the multilingual educational environment at FCIM. Performance on multilingual benchmarks confirms this capability, with the model maintaining consistent quality across language boundaries rather than exhibiting the performance degradation common in models primarily trained on English.

The model's handling of diacritical marks and language-specific orthography proves particularly important for serving diverse student populations. Official MMLU benchmark results demonstrate robust multilingual performance with scores ranging 60-65% across multiple language families, indicating strong cross-lingual understanding capabilities that enable consistent answer quality regardless of question language.

6 Performance Optimization

6.1 Latency Optimization

1. **Redis Caching**:
    - Cache frequent questions (TTL: 7 days)
    - Cache embeddings (TTL: 30 days)
    - Target hit rate: 30%+
    - Reduces latency by 95% for cached queries
2. **vLLM Optimizations**:
    - PagedAttention for efficient memory
    - Continuous batching
    - KV cache reuse
    - GPU utilization: 80%
3. **Short Generation**:
    - `max_tokens=50` (vs 512 in larger models)
    - Temperature=0.3 (focused, deterministic)
    - Typical answer: 1-5 words

6.2 Throughput Optimization

Several architectural decisions contribute to system throughput optimization. The FastAPI backend implements connection pooling to minimize overhead from repeated connection establishment. Asynchronous programming patterns using async/await are employed throughout the stack, enabling efficient handling of concurrent requests without thread blocking. The embedding service generates vectors in batches rather than individually, significantly improving throughput during both document ingestion and query processing. Finally, Qdrant's HNSW index provides logarithmic search complexity, ensuring that retrieval latency remains bounded even as the document collection grows.

7 Quality Metrics

7.1 Evaluation Methodology

The system employs RAG-specific evaluation metrics rather than traditional question-answering benchmarks, recognizing the unique characteristics of retrieval-augmented generation systems. Seven key performance indicators form the basis of quality assessment, spanning retrieval quality, generation accuracy, system performance, and operational efficiency. These metrics collectively ensure the system meets educational requirements while maintaining cost-effective operation.

Table 4: Key performance metrics

Metric	Description	Target
Retrieval Precision	Relevant docs in TOP-5	¿70%
Answer Relevance	Semantic similarity to question	¿0.65
Faithfulness	Answer based on context	¿0.80
Latency (p95)	95th percentile response time	¡1s
Throughput	Concurrent users supported	30-50
Cache Hit Rate	Queries served from cache	¿30%
NOT_ANSWERABLE Rate	Correctly identified	10-15%

7.2 Real-world Example

Course: Algorithms and Data Structures (ASD-2024)
Question: "What is the worst-case time complexity of quicksort?"
RAG Pipeline:

1. Embedding generation: 15ms
2. Qdrant search (TOP-5): 8ms
3. Context retrieved from `Lecture_02_Sorting.pdf`
4. Qwen2.5-3B generation: 180ms
5. Total latency: 203ms

Answer: "O(n^2) worst case"
Student satisfaction: Positive (Fast and accurate response)

8 Security & Privacy

8.1 Authentication

- JWT tokens (24h expiry)
- Integration with Moodle SSO
- Role-based access control

8.2 Rate Limiting

- 100 requests/minute per user
- 1000 requests/minute per IP
- Exponential backoff for violations

8.3 Data Privacy

- All data stored on-premises
- No external API calls
- Query logs encrypted at rest
- GDPR compliant

8.4 Input Validation

- Max question length: 2000 characters
- Sanitization of special characters
- SQL injection prevention
- XSS protection

9 Deployment

9.1 Docker Compose Orchestration

All system services are containerized and orchestrated through Docker Compose, providing consistent deployment across development, staging, and production environments. The orchestration configuration defines six primary services: the FastAPI backend, vLLM inference server, Qdrant vector database, embedding service, Redis cache, and monitoring stack (Prometheus and Grafana).

The deployment process begins by launching all services simultaneously in detached mode, allowing background operation while maintaining process isolation. Service health monitoring reveals the operational status of each container, including port mappings, resource consumption, and restart counts. Log aggregation from the backend service enables real-time debugging and performance analysis, with log levels configurable from debug through critical severity. For production scaling, the backend service supports horizontal replication, distributing load across multiple instances behind an internal load balancer. A typical production deployment runs three backend replicas to ensure high availability and load distribution during peak usage periods.

9.2 Service Configuration

The vLLM inference server operates with carefully tuned parameters optimizing the trade-off between throughput and resource utilization. The server binds to all network interfaces on port 8001, enabling access from other containerized services within the Docker network. Maximum sequence length is constrained to 4096 tokens, balancing context capacity with memory efficiency for the 3B parameter model. GPU memory utilization targets 80% allocation, reserving headroom for attention mechanism overhead and preventing out-of-memory errors during peak loads. The automatic datatype selection enables mixed-precision inference, utilizing FP16 where appropriate while maintaining FP32 for critical operations requiring numerical precision.

The FastAPI backend server configuration emphasizes concurrent request handling and observability. The server exposes endpoints on port 8000 across all network interfaces, accepting connections from the Moodle frontend and administrative tools. Four worker processes enable parallel request processing, each handling independent request streams without blocking. The info-level logging captures essential operational events including request arrivals, response times, and error conditions, while excluding verbose debug output that would overwhelm log storage. This configuration supports the target throughput of 30-50 concurrent users while maintaining sub-second p95 latency.

10 Monitoring & Observability

10.1 Metrics Collection

Prometheus continuously collects performance metrics from all system components, providing comprehensive observability into system behavior. FastAPI exports metrics tracking request volumes, latency distributions, and error rates, enabling detection of API-level issues. The vLLM inference server reports GPU utilization and token generation throughput, critical for capacity planning and performance optimization. Redis cache metrics including hit rates and memory usage guide caching strategy refinement. Finally, Qdrant provides search latency and index size statistics, ensuring retrieval performance remains within acceptable bounds as the document collection expands.

10.2 Grafana Dashboards

Grafana dashboards visualize collected metrics across multiple operational perspectives. The System Overview dashboard presents high-level health indicators including latency trends, throughput patterns, and error rates, enabling rapid assessment of overall system status. GPU-specific dashboards track utilization, memory consumption, and temperature, critical for preventing hardware issues and optimizing resource allocation. Cache Performance dashboards monitor hit rates and memory usage, informing decisions about cache sizing and eviction policies. User Activity dashboards aggregate query patterns including queries per hour and course access distributions, providing insights into usage patterns that guide system optimization and capacity planning.

10.3 Health Monitoring

The system exposes two health check endpoints serving different operational needs. The primary health endpoint provides a lightweight binary status indicator, responding with HTTP 200 when all critical services are operational or HTTP 503 during degraded states. This endpoint supports load balancer health probes and uptime monitoring systems requiring simple pass/fail status without detailed diagnostics.

The detailed health endpoint returns comprehensive component-level status information in JSON format. This endpoint reports individual health states for the vLLM inference server, Qdrant vector database, Redis cache, and embedding service. Each component status includes operational metrics such as response times, error rates, and resource utilization levels. The detailed endpoint supports operational debugging, capacity planning, and automated alerting based on component-specific thresholds. Production

monitoring systems query this endpoint every 30 seconds, triggering alerts when any component reports degraded or failed status.

11 Review of Completed Work and Future Research Directions

11.1 Accomplished Development and Research Activities

This section provides a comprehensive review of completed research, development, and optimization work conducted during the project lifecycle.

11.1.1 Phase 1: Systematic Model Evaluation Campaign

We conducted a systematic comparative evaluation of 24 state-of-the-art language models spanning multiple model families (Qwen, Mistral, Llama, Yi, CodeQwen) on RAG-specific question answering tasks. Evaluation utilized standard benchmarks including SQuAD v2.0 (Stanford Question Answering Dataset) and HotpotQA for multi-hop reasoning assessment.

Key Research Findings:

Our systematic evaluation revealed four critical insights that fundamentally shaped the project direction. First, we identified catastrophic methodological issues in prompt engineering that caused exact match performance to collapse to 0-1% across all evaluated models, regardless of their size or capabilities. This finding highlighted the critical importance of task-specific prompt optimization in RAG applications. Second, we discovered that the commonly referenced "Combined Score" composite metric, which aggregates multiple evaluation dimensions, is non-standard in the literature and unsuitable for cross-study performance comparison, necessitating adoption of established metrics from the RAGAS framework. Third, we established empirical evidence that smaller language models with 3 billion parameters achieve 80-85% of the answer quality produced by models exceeding 32 billion parameters, while requiring only 10% of the computational resources. Finally, we documented significant retrieval precision consistency at approximately 70% across all evaluated models, demonstrating that retrieval quality is fundamentally independent of generative model size and depends primarily on embedding model quality and retrieval strategy.

11.1.2 Phase 2: Evaluation Methodology Refinement

Following the discovery of systematic evaluation issues, we implemented five critical methodological improvements that substantially enhanced assessment reliability. First, we reformulated generation instructions to enforce brevity constraints through explicit "Answer in 1-5 words only" prompting, effectively reducing verbosity-induced evaluation metric degradation. Second, we reduced the sampling temperature parameter from 0.7 to 0.3, decreasing output stochasticity and significantly improving answer consistency across repeated evaluations.

Third, we constrained maximum token generation from 512 to 50 tokens, enforcing concise answer generation that aligns with educational question-answering requirements where students expect direct responses rather than lengthy explanations. Fourth, we implemented semantic Answer Relevance scoring using SentenceTransformer-based cosine

similarity between generated answers and ground truth, providing substantially more robust quality assessment than exact string matching which penalizes paraphrases and synonyms. Finally, we established an explicit "NOT_ANSWERABLE" response protocol for questions lacking sufficient context, addressing a major weakness in SQuAD v2.0 evaluation where 65% of questions have no valid answer in the provided context.

11.1.3 Phase 3: Architecture Optimization and Model Selection

Based on comprehensive benchmarking results, we made a strategic architectural decision to deploy Qwen2.5-3B-Instruct rather than the initially planned Qwen2.5-32B-Instruct configuration. This decision was justified by the following quantitative analysis:

Table 5: Comparative Analysis: Qwen2.5-32B vs Qwen2.5-3B

Metric	**32B Model**	**3B Model**
GPU Requirements	64GB (A100)	6GB (T4)
Cost Reduction	Baseline	90% savings
Inference Latency (median)	800-1500ms	150-300ms
Latency Improvement	Baseline	5× faster
Concurrent User Capacity	5-10 users	30-50 users
Throughput Improvement	Baseline	5× higher
Answer Relevance (semantic)	75-80%	65-70%
Quality Degradation	Baseline	-10% acceptable

The analysis demonstrates that for educational chatbot applications where response latency and system availability are prioritized over marginal quality improvements, the 3B model configuration provides optimal cost-performance characteristics.

11.2 Planned Future Development

11.2.1 Short-term Research Objectives (Q1 2026)

Several immediate enhancements are planned for the first quarter of 2026. Response streaming through server-sent events (SSE) will enable progressive answer generation, improving perceived latency for longer responses by displaying partial results as they are generated rather than waiting for complete responses. This enhancement particularly benefits questions requiring more elaborate explanations.

Multimodal capabilities represent another priority, extending the system to extract and process images and diagrams from PDF documents. Vision-language model integration will enable visual question answering, allowing students to ask questions about charts, diagrams, and mathematical notation that currently remain inaccessible to text-only processing.

Domain adaptation through fine-tuning Qwen2.5-3B on course-specific corpora using Parameter-Efficient Fine-Tuning methods such as LoRA and QLoRA promises improved answer quality without the computational overhead of full model retraining. Finally, implementing an explicit feedback collection mechanism with binary quality ratings will enable continuous quality monitoring and support reinforcement learning from human feedback (RLHF), creating a continuous improvement loop driven by actual student interactions.

11.2.2 Medium-term Research Objectives (Q2-Q3 2026)

The second and third quarters will focus on enhancing system sophistication and reach. Conversational memory implementation will transform the system from single-turn interactions to multi-turn dialogues, maintaining conversation history and context across multiple exchanges. This capability is essential for complex pedagogical interactions where questions naturally build upon previous responses.

Source attribution enhancements will add automatic citation generation with precise page numbers and document references, enabling students to verify information and explore topics in greater depth. This transparency builds trust and encourages critical engagement with source materials rather than passive acceptance of generated answers.

Mobile application development will extend system accessibility through native iOS and Android clients featuring offline caching capabilities. Mobile-first design recognizes contemporary student behavior patterns where smartphones serve as primary learning devices. Finally, establishing a systematic A/B testing framework will enable rigorous prompt and parameter optimization with statistical significance testing, ensuring improvements are data-driven rather than anecdotal.

11.2.3 Long-term Research Directions (Q4 2026 and Beyond)

Longer-term research directions envision system evolution beyond single-institution deployment. Multi-institutional scaling will extend the system to other universities, requiring development of an institution-specific customization framework that accommodates varying pedagogical approaches, document formats, and organizational structures while maintaining core functionality.

Content generation capabilities will transform the system from purely reactive to proactive, automatically generating quiz and assessment questions from course materials using controllable generation techniques. This functionality supports formative assessment and self-directed learning by providing students with practice materials aligned with actual course content.

Adaptive learning integration represents a significant architectural evolution, developing personalized recommendation systems based on student interaction patterns and identified knowledge gaps. By analyzing question histories and performance indicators, the system could guide students toward materials addressing their specific learning needs.

Finally, edge deployment research will investigate on-device model deployment for privacy-sensitive scenarios, employing quantization and model compression techniques to enable local inference without compromising student data privacy. This direction acknowledges growing privacy concerns and regulatory requirements while maintaining system utility.

12 Conclusion

This technical report presented the design, implementation, and evaluation of a multilingual RAG-based educational chatbot for the FCIM ELSE platform. Through systematic experimentation and optimization, we achieved a production-ready system with the following key outcomes:

12.1 Quantitative Results

- **Performance**: Sub-second response latency (p95 $<$ 1s) with 150-300ms median
- **Quality**: 70%+ retrieval precision, 65-70% answer relevance on evaluation datasets
- **Cost Efficiency**: 90% operational cost reduction through optimal model selection (Qwen2.5-3B vs 32B)
- **Scalability**: Production capacity of 30-50 concurrent users on commodity hardware (Nvidia T4)
- **Multilingual Support**: Native processing capabilities for required language set

12.2 Key Findings

Our experimental evaluation yielded several important insights:

Model Size vs. Performance The widely assumed necessity of large language models (30B+ parameters) for high-quality RAG applications was challenged by our results. Qwen2.5-3B-Instruct demonstrated that smaller, well-optimized models can achieve 80-85% of larger model quality while providing 5$\times$ improvements in latency and throughput.

Evaluation Methodology We identified critical issues in standard RAG evaluation practices, including inappropriate prompt verbosity and non-standard composite metrics. Our corrected methodology using Answer Relevance (semantic similarity), Retrieval Precision, and explicit unanswerable question handling provides more reliable quality assessment.

Cost-Performance Tradeoffs For educational chatbot applications where response speed and availability are prioritized over marginal quality improvements, aggressive optimization toward smaller models proves highly effective. The 10% quality reduction is acceptable given 90% cost savings and 5$\times$ latency improvement.

12.3 Limitations and Future Work

The current implementation exhibits several limitations that inform future development priorities. The system operates exclusively in single-turn conversation mode, processing each question independently without maintaining dialogue history or contextual awareness across interactions. Processing remains limited to text-only content, leaving images and diagrams embedded in course materials inaccessible to students seeking visual information. The deployed model has not undergone course-specific fine-tuning, relying instead on general-purpose language understanding that may miss domain-specific terminology and conventions. Finally, while the system demonstrates strong multilingual capabilities, expansion to additional languages requires careful evaluation of model performance and embedding quality before deployment.

Future development will systematically address these limitations through the planned enhancements outlined in the previous section, including streaming responses for improved user experience, multimodal integration for comprehensive document understand-

ing, domain-specific fine-tuning for enhanced accuracy, and expanded language support to serve increasingly diverse student populations.

12.4 Practical Impact

The implemented system demonstrates that effective educational AI assistants can be deployed at modest cost using modern RAG architectures and carefully selected smaller language models. This approach makes AI-powered educational support accessible to institutions with limited computational budgets, potentially benefiting thousands of students across multiple universities.

Acknowledgments

This work was conducted at the Faculty of Computers, Informatics and Microelectronics, Technical University of Moldova. We thank the ELSE platform development team for their collaboration and the students who provided feedback during system testing.

References

1. Qwen Team. (2024). *Qwen2.5: A Party of Foundation Models*. https://qwenlm.github.io/blog/qwen2.5/

2. Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020.

3. vLLM Team. (2024). *vLLM: Easy, Fast, and Cheap LLM Serving*. https://github.com/vllm-project/vllm

4. Es, S., et al. (2023). *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. arXiv:2309.15217

5. Qdrant Team. (2024). *Qdrant Vector Search Engine Documentation*. https://qdrant.tech/documentation/